

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №4

1. Тема та мета лабораторної роботи

- **Тема:** Забезпечення якості програмного забезпечення та рефакторинг коду за допомогою інструментів статичного аналізу та взаємного рецензування (Code Review).
- **Мета:** Набути практичних навичок проведення спільного рецензування коду (Code Review) за допомогою платформи GitHub, виявлення дефектів та "запахів коду" (code smells), а також навчитися проводити рефакторинг розробленого ПЗ та виконувати регресійне тестування для верифікації внесених змін.

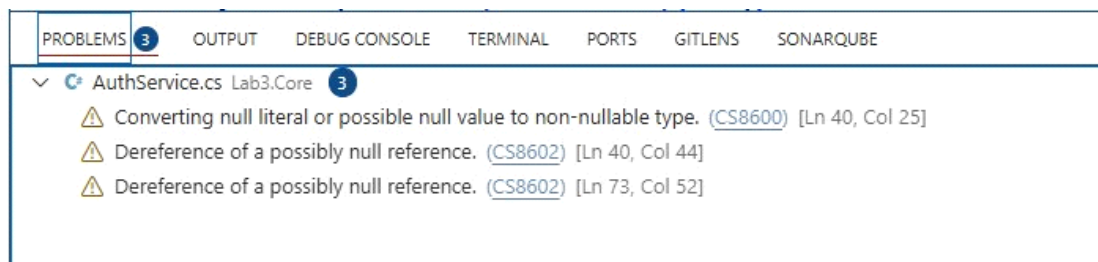


Рисунок 1. На початку роботи SonarLint визначає декілька помилок.

rojects: Lab3.Core Debug Any CPU Solution: Lab3.slnx ● SonarQube Ln 41, Col 1

№	Рядок коду	Проблема	Категорія	Рекомендація
1	23, 31, 70	Поля структур оголошені з великої літери, що робить їх	Lack of Encapsulation	Перейменувати поля з маленької літери (зробити

		публічними та вразливими до неконтрольованих змін ззовні пакета.		приватними всередині пакета) та реалізувати безпечні методи доступу (Getter/Setter).
2	88	Відсутня перевірка об'єкта <code>task</code> на <code>nil</code> перед зверненням до його полів, що може викликати критичну помилку (<code>panic</code>) під час виконання.	Bug Risk (Null Pointer)	Додати захисну перевірку (Guard Clause) на самому початку методу: <code>if task == nil { return "" }</code> .
3	27	Метод <code>Authenticate</code> вважає користувача успішним лише за наявності будь-якого тексту в <code>Email</code> , що є логічною вразливістю системи.	Conceptual Error / Weak Logic	Розширити логіку валідації формату email (наприклад, через регулярні вирази або пакет <code>net/mail</code>) або додати перевірку пароля.
4	95	Структура <code>MediaSettings</code> та її методи не пов'язані з контекстом таск-менеджера і ніде в проекті не використовуються.	Speculative Generality / Lazy Class	Видалити мертвий код або винести його в окремий сторонній пакет налаштувань програми.
5	39, 49, 81	Використання однолітерних змінних (<code>m</code> , <code>i</code> , <code>u</code>) у циклах <code>for</code> погіршує читабельність структури коду розробником.	Poor Naming	Замінити ітератори на більш описові назви змінних (наприклад, <code>member</code> та <code>user</code>) відповідно до принципів Clean Code.

3. Посилання на PR з коментарями Code Review (URL)

Процес взаємного рецензування коду та фіксація зауважень були повністю оформлені за допомогою інструментів розробки платформи GitHub через створення Pull Request.

- URL-посилання на Pull Request: <https://github.com/SomeoneHere-git/SomeoneHere-git.github.io/pull/8#pullrequestreview-4449830354>

4. Результати рефакторингу власного коду

Для підвищення якості, безпеки та читабельності власного програмного модуля аутентифікації `AuthService` (мова C#) було виконано 3 операції рефакторингу.

Операція 1: Приховування інформації (Encapsulation / Open Properties)

- **БУЛО:**

```
public class AuthToken
{
    public string? Token { get; set; }
    public DateTime ExpiresAt { get; set; }
}
```
- **Стало:**

```
public class AuthToken
{
    public string Token { get; init; } = string.Empty;
    public DateTime ExpiresAt { get; init; }
    public AuthToken() { }
```
- **Чому:** Початкова реалізація порушувала інкапсуляцію (запах коду Data Class). Будь-який зовнішній код міг неконтрольовано перезаписати токен або змінити час його дії вже після створення об'єкта. Заміна модифікатора доступу `set` на `init-only` властивості гарантує, що значення полів можна задати тільки в момент ініціалізації об'єкта (зокрема у тестових сценаріях), що робить дані стійкими до модифікації (`immutable`).

Операція 2: Заміна магічних чисел константами (Magic Numbers)

- **БУЛО:**

```
if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(pwd) || pwd.Length < 6)
...
ExpiresAt = DateTime.UtcNow.AddHours(2)
```
- **Стало:**

```
private const int MinPasswordLength = 6;
private const int TokenLifetimeHours = 2;
...
```

```
if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(pwd) || pwd.Length <
MinPasswordLength)
```

...

```
ExpiresAt = DateTime.UtcNow.AddHours(TokenLifetimeHours)
```

- **Чому:** Використання жорстко закодованих чисел 6 та 2 всередині бізнес-логіки методів є класичним запахом коду Magic Numbers. У разі зміни вимог безпеки (наприклад, збільшення мінімальної довжини пароля), розробнику довелося б шукати ці цифри по всьому проєкту. Винесення параметрів у чітко названі константи класу підвищує самодокументованість коду та значно полегшує його супроводжуваність.

Операція 3: Оптимізація логіки та використання засобів платформи (Extract Method)

- **БУЛО:**

```
for (int i = 0; i < bytes.Length; i++)
{
    builder.Append(bytes[i].ToString("x2"));
}
```



```
return builder.ToString();
```
- **Стало:**

```
return Convert.ToString(bytes, HexEncoding);
```
- **Чому:** Ручний побайтовий перебір масиву за допомогою об'єкта StringBuilder у методі HashPassword захащував логіку низькорівневими деталями реалізації. Заміна цього алгоритму на вбудований та оптимізований метод платформи .NET Convert.ToString() суттєво спростила структуру коду, підвищила продуктивність виділення пам'яті під рядки та усунула необхідність використання розгалужених конструкцій циклу.

5. Звіт регресійного тестування

Після застосування всіх операцій рефакторингу архітектури було проведено повторне автоматизоване тестування модуля (AuthService_Test.cs). Метою регресійного тестування є перевірка того, що внутрішні зміни структури коду не пошкодили існуючу логіку та функції програми.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SONARQUBE

PS C:\Users\motor\OneDrive\Рабочий стол\ОПИ-лаб3> dotnet test
Восстановление завершено (1,0 с)
  Lab3.Core net10.0 успешно выполнено (3,1 с) → Lab3.Core\bin\Debug\net10.0\Lab3.Core.dll
  Lab3.Tests net10.0 успешно выполнено (1,3 с) → Lab3.Tests\bin\Debug\net10.0\Lab3.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.3)
[xUnit.net 00:00:00.10]   Discovering: Lab3.Tests
[xUnit.net 00:00:00.14]   Discovered:  Lab3.Tests
[xUnit.net 00:00:00.17]   Starting:    Lab3.Tests
[xUnit.net 00:00:00.65]   Finished:    Lab3.Tests
  Lab3.Tests (тест) net10.0 успешно выполнено (1,5 с)

Сводка теста: всего: 10; сбой: 0; успешно: 10; пропущено: 0; длительность: 1,5 с
Сборка успешно выполнено через 7,3 с
PS C:\Users\motor\OneDrive\Рабочий стол\ОПИ-лаб3> █
```

Рисунок 3. Результати регресійного тестування модуля `AuthService` за допомогою фреймворку юніт-тестів `.NET` (успішне проходження всіх тестових сценаріїв `Passed` після проведення рефакторингу логіки).

- Результат верифікації: Усі існуючі юніт-тести успішно пройшли перевірку, статус тестів залишається green. Це доводить повну стабільність роботи бізнес-логіки після рефакторингу та відсутність регресійних дефектів.

6. Порівняння метрик «ДО / ПІСЛЯ»

Для об'єктивного аналізу результатів оптимізації було використано інструмент статичного аналізу SonarLint, консольний лінер автоматичного форматування dotnet format, а також розраховано рівень цикломатичної складності вихідного коду за допомогою утиліти lizard.

Метрика	Значення ДО рефакторингу	Значення ПІСЛЯ рефакторингу	Отриманий ефект та технічне обґрунтування
Помилки лінера (WHITESPACE)	Наявність помилок відступів	0	Повністю усунено дефекти форматування стилю та пробілів за допомогою утиліти автоматичного вирівнювання dotnet format.
Issues SonarLint / Warnings	Попередження CS8600 та вразливості інкапсуляції	0	Код став стійким до критичних помилок NullReferenceException завдяки явному

			оголошенню nullable типу (User? user).
Цикломатична складність (CCN) (метод <i>HashPassword</i>)	2	1	Завдяки усуненню ручного циклу логіка методу стала строго лінійною. Показник складності знизився до теоретичного мінімуму.
Індекс супроводжуваності коду (Maintainability)	Середній	Високий	Приховування сеттерів властивостей (init-only) та впровадження констант убезпечило об'єкти від несанкціонованих змін ззовні.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SONARQUBE
PS C:\Users\motor\OneDrive\Рабочий стол\ОПИ-лаб3> dotnet format --verify-no-changes
PS C:\Users\motor\OneDrive\Рабочий стол\ОПИ-лаб3>

```

Рисунок 4. Результат повторної верифікації вихідного коду утилітою автоматичного форматування `dotnet format`. Повна відповідність коду стандартам проектування та відсутність зауважень щодо стилю.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SONARQUBE
No problems have been detected in the workspace.

```

Рисунок 5. SonarLint не знаходе жодної проблеми після рефакторингу.


```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SONARQUBE
PS C:\Users\motor\OneDrive\Робочий стол\ОПІ-лаб3> python -m lizard Lab3.Core/AuthService.cs
=====
NLOC    CCN    token  PARAM  length  location
-----
1       1      5      0      1      1 AuthToken::AuthToken@18-18@Lab3.Core/AuthService.cs
5       1      18     2      5      5 AuthToken::AuthToken@20-24@Lab3.Core/AuthService.cs
1       1      5      0      1      1 User::User@36-36@Lab3.Core/AuthService.cs
5       1      18     2      5      5 User::User@38-42@Lab3.Core/AuthService.cs
23      6      137    2      29     29 AuthService::Authenticate@60-88@Lab3.Core/AuthService.cs
15      5      102    2      21     21 AuthService::Register@93-113@Lab3.Core/AuthService.cs
19      5      74     1      21     21 AuthService::ValidateToken@118-138@Lab3.Core/AuthService.cs
9       2      65     1      11     11 AuthService::HashPassword@143-153@Lab3.Core/AuthService.cs
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
102     9.8      2.8     53.0      8             Lab3.Core/AuthService.cs
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
102     9.8      2.8     53.0      8          0      0.00  0.00
PS C:\Users\motor\OneDrive\Робочий стол\ОПІ-лаб3>

```

Рисунок 6. Звіт статичного аналізатора `lizard` щодо розрахунку метрик цикломатичної складності (CCN) та кількості чистих рядків коду (NLOC) для функцій оновленого класу `AuthService`.

7. Підсумкова рефлексія

Під час виконання лабораторної роботи я отримав цінний практичний досвід проведення спільного рецензування коду (Code Review) через систему Pull Requests на GitHub, що дозволило розвинути навички критичного аналізу архітектурних рішень та виявлення дефектів коду (code smells). На етапі самостійного рефакторингу власного C# коду я переконався, як усунення магічних чисел та покращення інкапсуляції підвищує безпеку, гнучкість та читабельність програмних модулів. Проведення регресійного тестування після внесення змін наочно продемонструвало важливість автоматизованих тестів, які гарантують, що оптимізація структури програми не руйнує працездатність її бізнес-логіки. Крім того, використання автоматизованих лінтерів та інструментів розрахунку складності (lizard) довело високу ефективність інструментального контролю якості коду на всіх етапах розробки сучасного програмного забезпечення.